Python Data Analysis Kickstart

Configuring Visual Studio code

Some settings to make programming with Python easier

Auto-save

- Search for "auto save"
- Set to *after delay*

Launch folder

- Search for "execute in"
- Check box for **Python > Terminal: Execute in File Dir**

Minimap

- Search for "minimap enabled"
- Uncheck **Editor > Minimap: Enabled**

Editor font size

- Search for "editor font size"
- Set **Editor: Font Size** to desired size

Terminal font size

- Search for "terminal font size"
- Set **Terminal: Font Size** to desired size

Themes

- Got to **File > Preferences > Theme > Color Theme**
- Select new theme as desired

Creating Variables

```
x = 5
```

Creating Variables

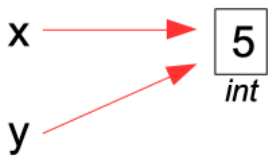
```
x = 5
```

Creating Variables

```
x = 5
y = x
```

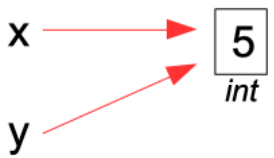
Creating Variables

```
x = 5
y = x
```



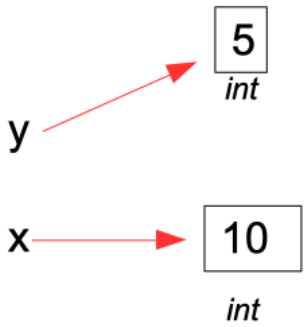
Creating Variables

```
x = 5
y = x
x = 10
```



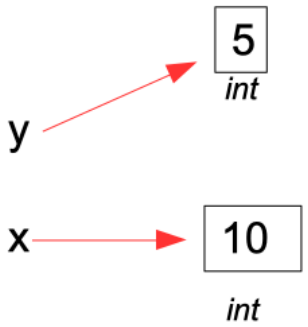
Creating Variables

```
x = 5
y = x
x = 10
```



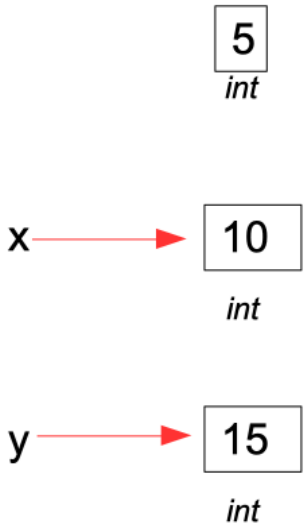
Creating Variables

```
x = 5
y = x
x = 10
y = 15
```



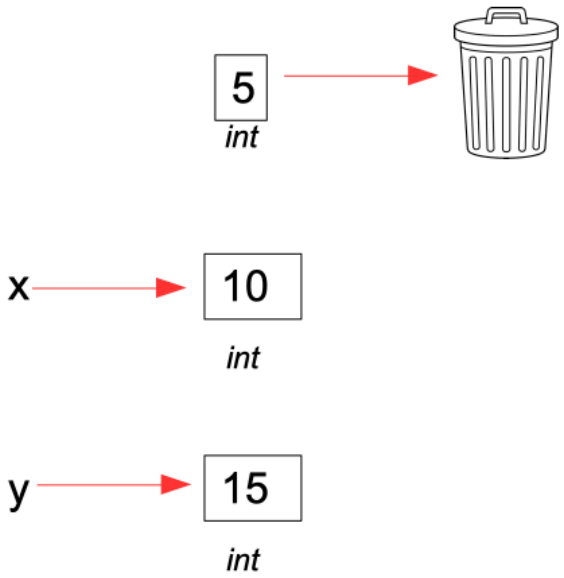
Creating Variables

```
x = 5
y = x
x = 10
y = 15
```



Creating Variables

```
x = 5
y = x
x = 10
y = 15
```



String literals

- Three flavors
  - single-delimited
  - triple-delimited
  - raw

Single-delimited

- Use either single or double quote character

```
"spam\n"
'spam\n'

print("Guido's the bomb!")
print('Guido is the "benevolent" dictator of Python')
```

Triple-delimited

- Single or double quote character
- No need to escape quotes

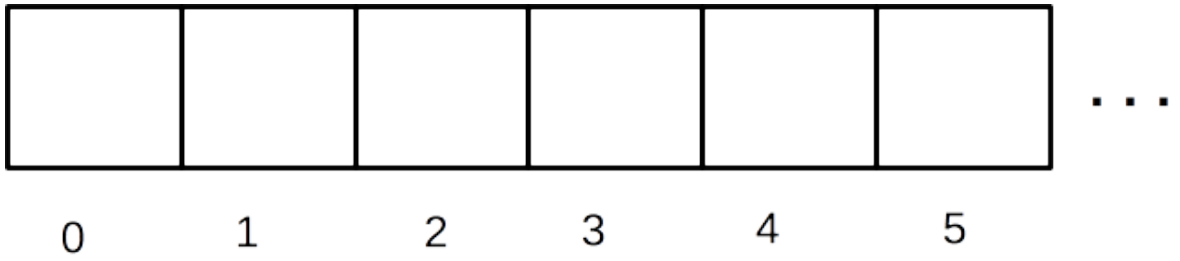
```
"""spam\n"""
'''spam\n'''

query = """
    select *
    from logs
    where date > '2018-02-19'
"""

print('''Guido's the "benevolent" dictator of Python''')
```

Raw

- Does not interpret backslashes

```
r"spam\n"
r'spam\n'
```

Sequences




```
colors = ['purple', 'orange', 'black']
print(colors[1])   # prints 'orange'
for color in colors:
    print(color)
```

Slices


```
s = "WOMBAT"

s[0:3]       # first 3 characters  "WOM"
s[:3]        # same, using default start of 0 "WOM"
s[1:4]       # s[1] through s[3] "OMB"
s[3:6]       # s[3] through end  "BAT"
s[3:len(s)]  # s[3] through end  "BAT"
s[3:]        # s[3] through end, using default end "BAT"
```

Dictionary

- Key/value pairs
- Keys must be immutable
  - str
  - int, float
  - tuple
- Keys are unique
- Keys/values stored in insertion order

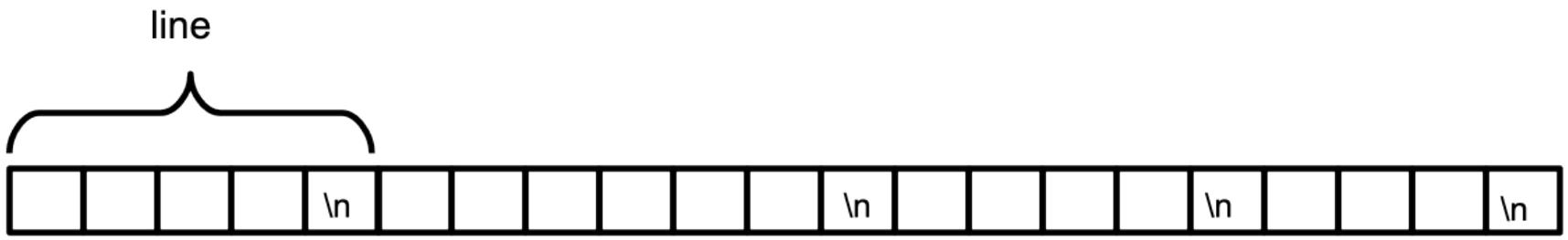
Dictionary items

```
for key, value in _DICT_.items():
    ...  # use key or value here
```

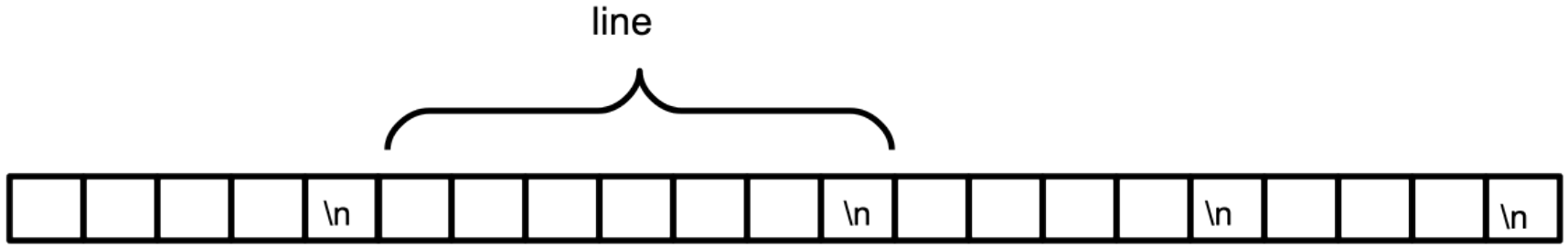
Reading Text Files


```
with open("somefile") as file_in:
```

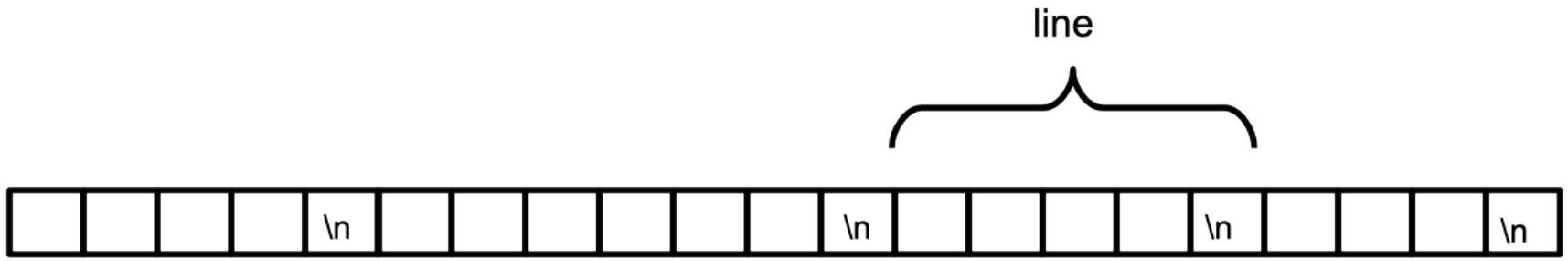
Reading one line at a time



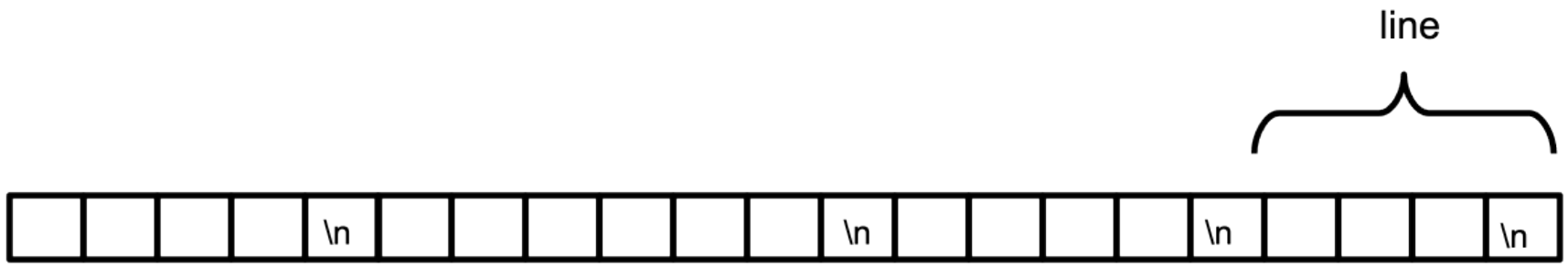
Reading one line at a time



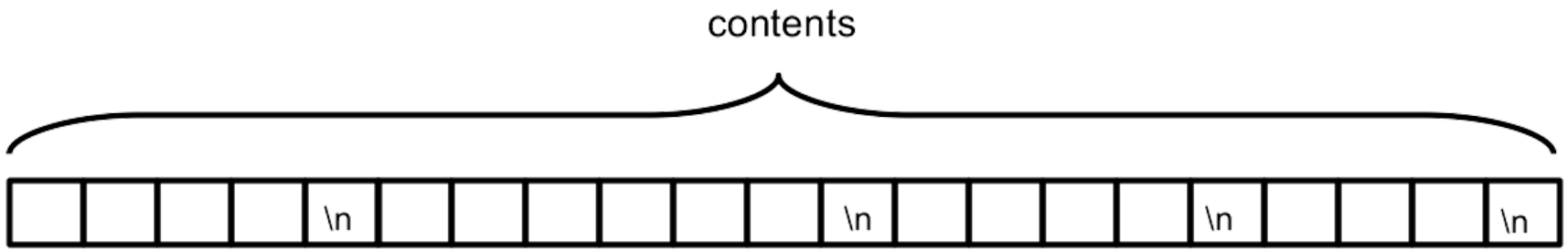
Reading one line at a time



Reading one line at a time

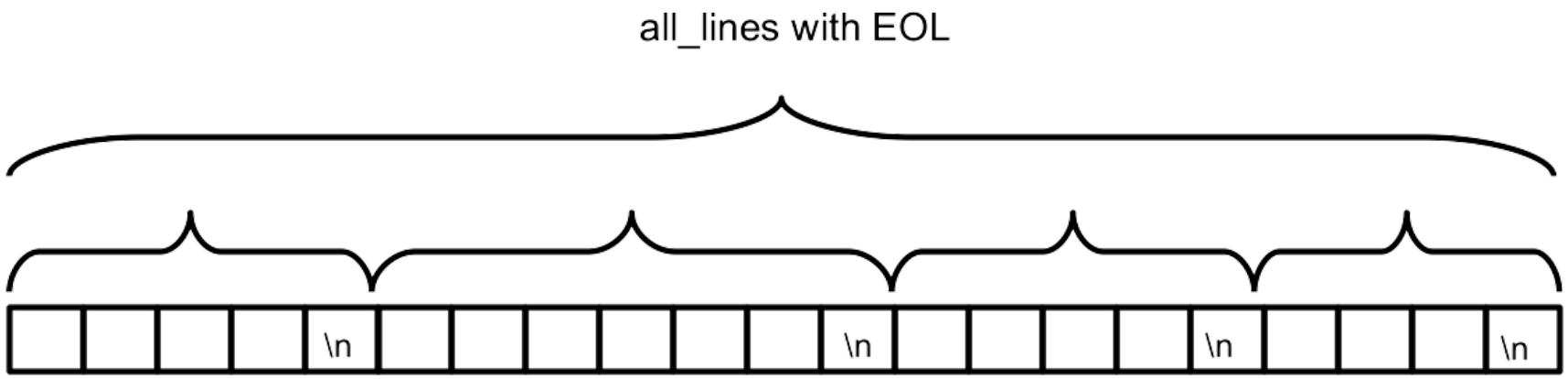


Reading entire file into string



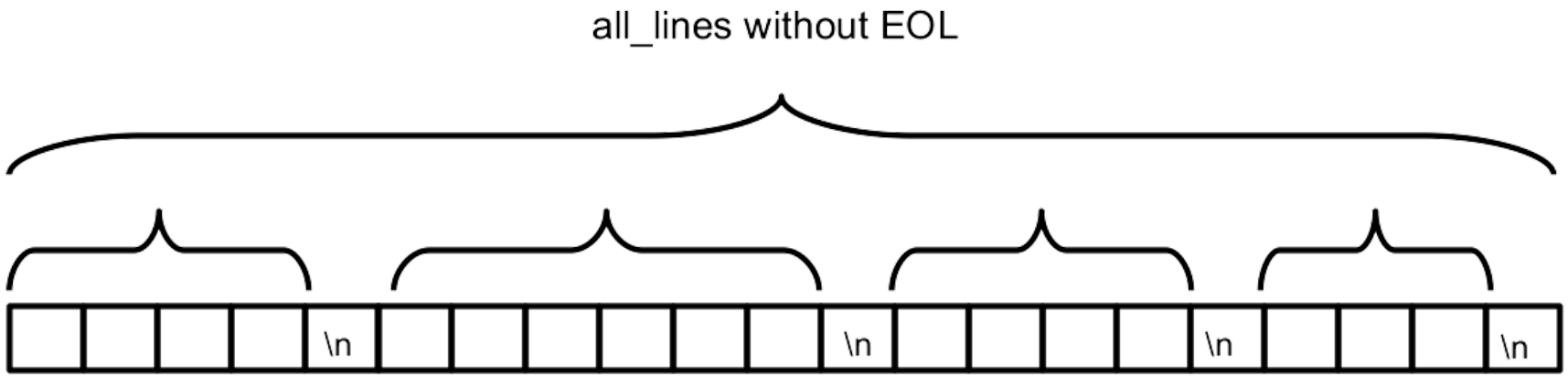

```
with open("somefile") as file_in:
    contents = file_in.read()
```

Reading file into list of strings (with EOL)



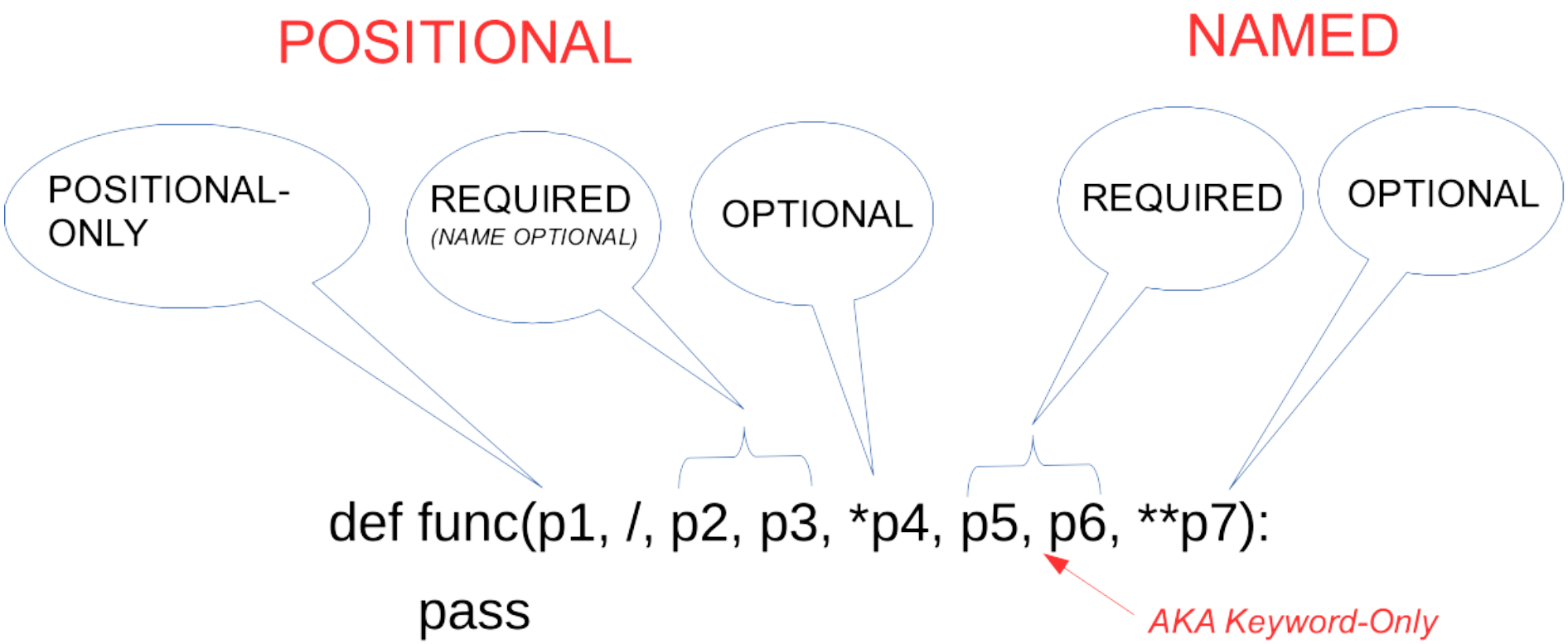

```
with open("somefile") as file_in:
    all_lines = file_in.readlines()
```

Reading file into list of strings (without EOL)

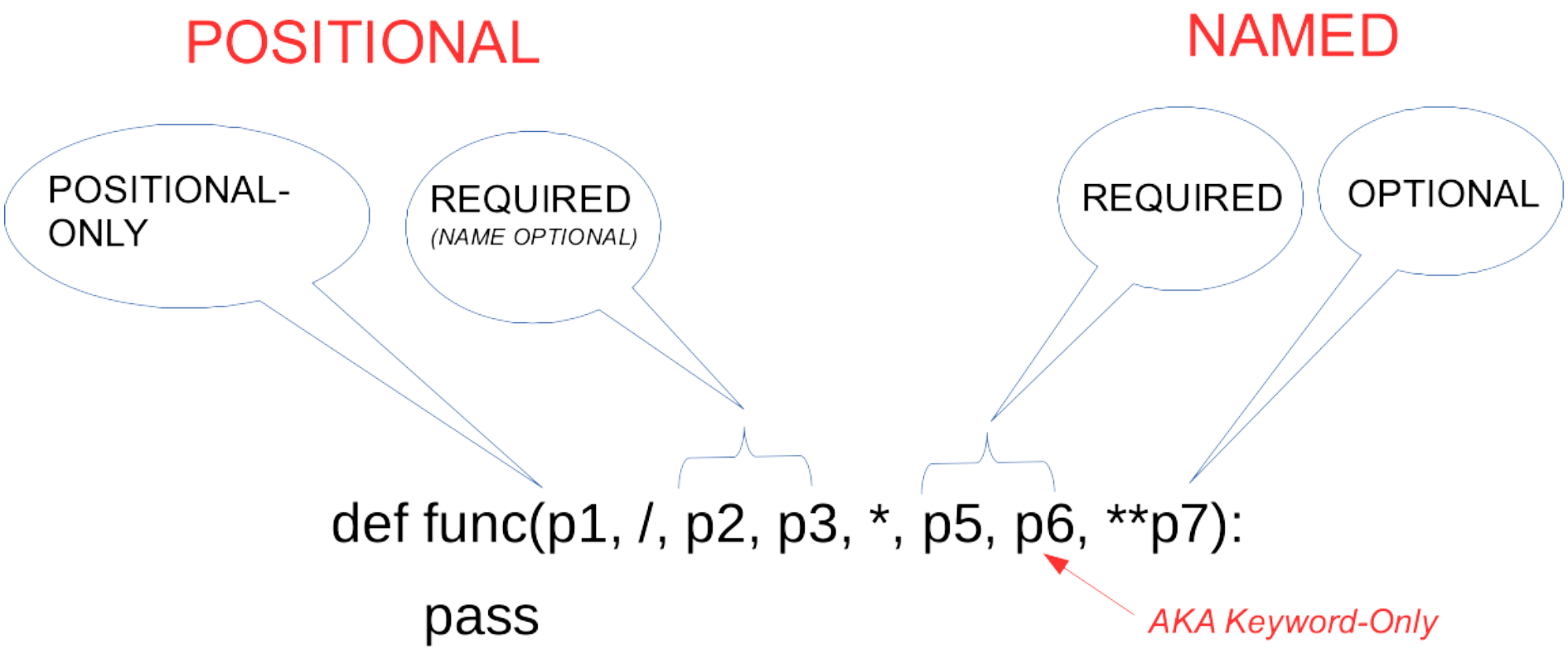



```
with open("somefile") as file_in:
    all_lines = file_in.read().splitlines()
```

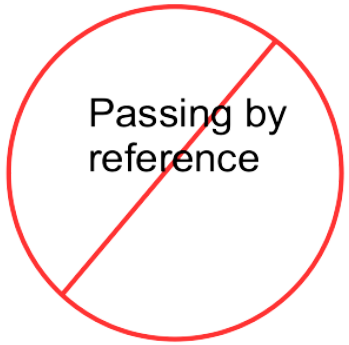
Function parameters



Function parameters



Argument passing

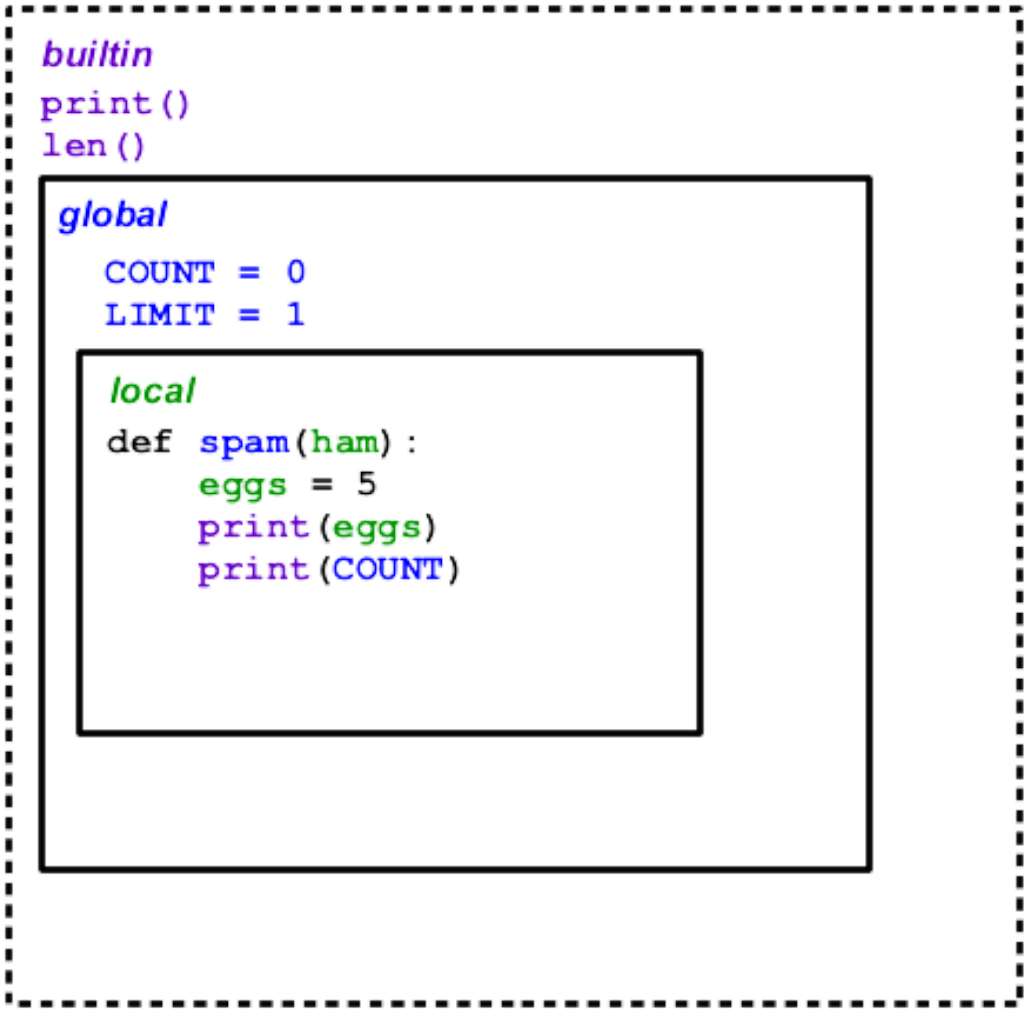


Passing by
sharing

- Read-only reference is passed
- Mutables may be changed via reference
- Immutables may not be changed

```
def spam(x, y):
    x = 5
    y.append("ham")

foo = 17
bar = ["toast", "jam"]

spam(foo, bar)
```

Variable Scope



Variable scope

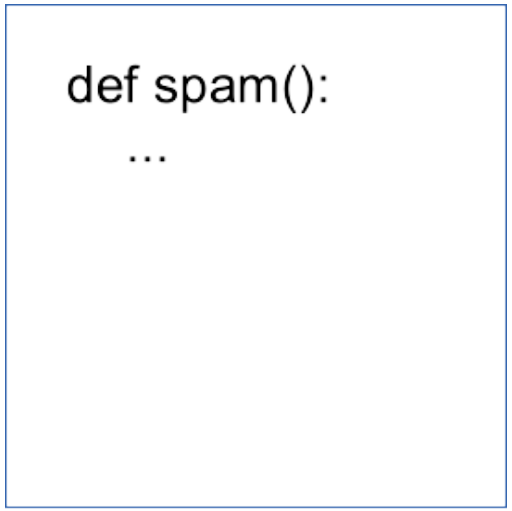
```
ALPHA = 10

def spam(beta):
    gamma = 20
    print(ALPHA)
    print(beta)
    print(gamma)

spam(1234)
```

BUILTIN

GLOBAL

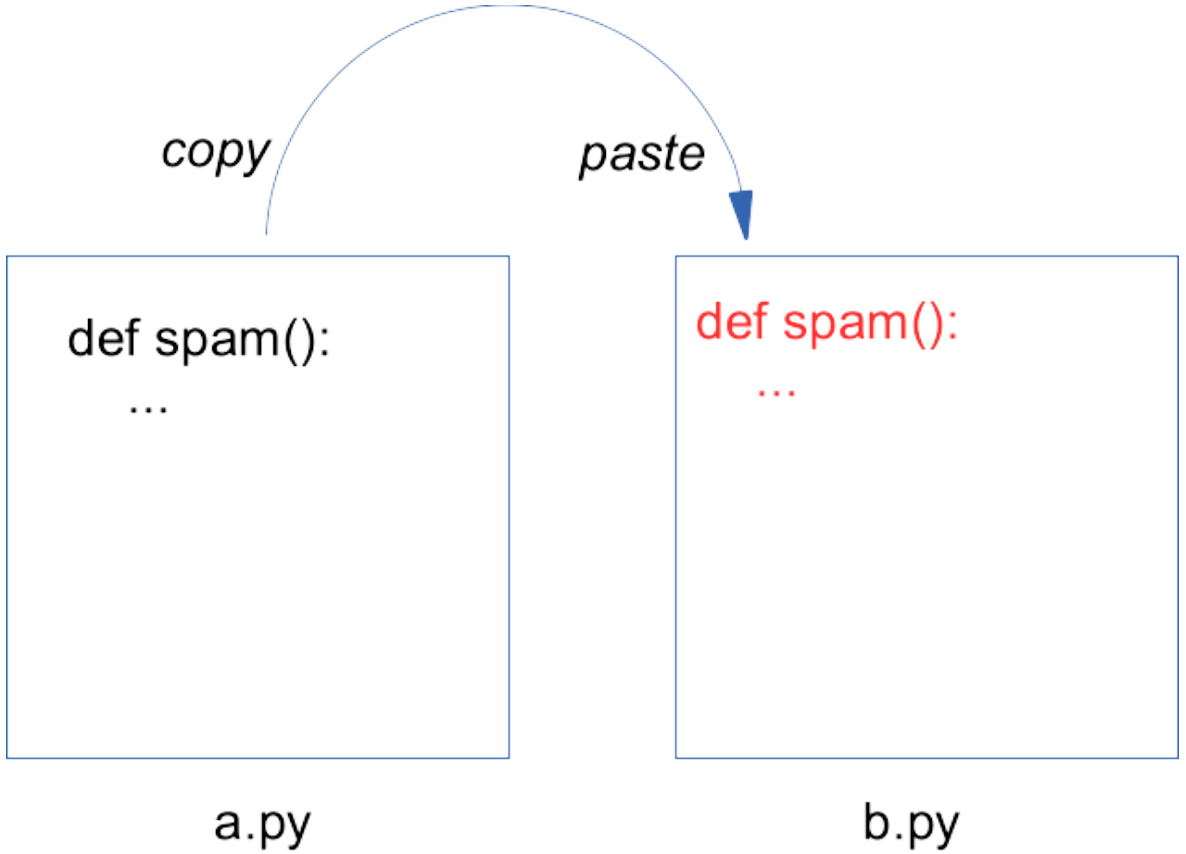
LOCAL

Copy/pasting functions

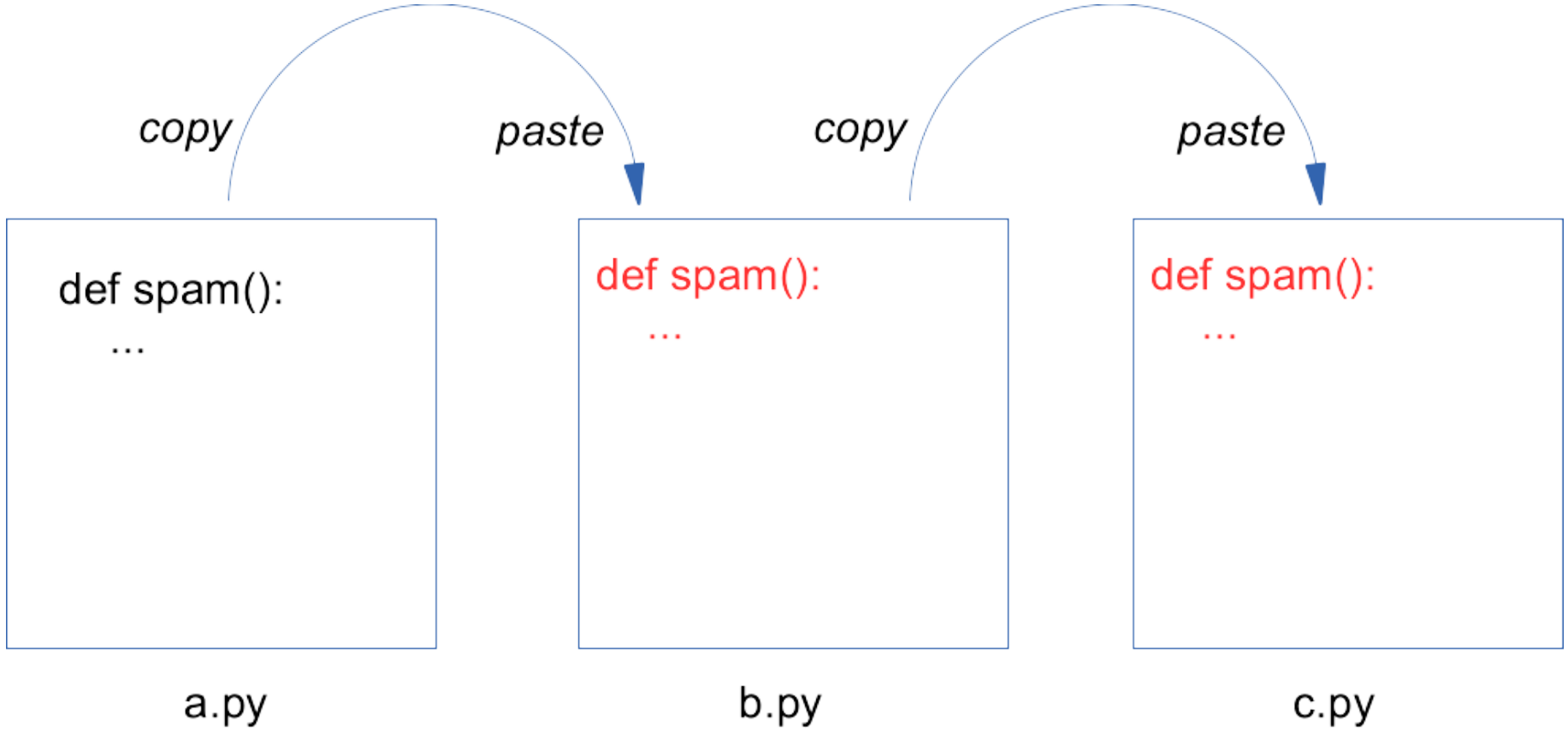


a.py

Copy/pasting functions

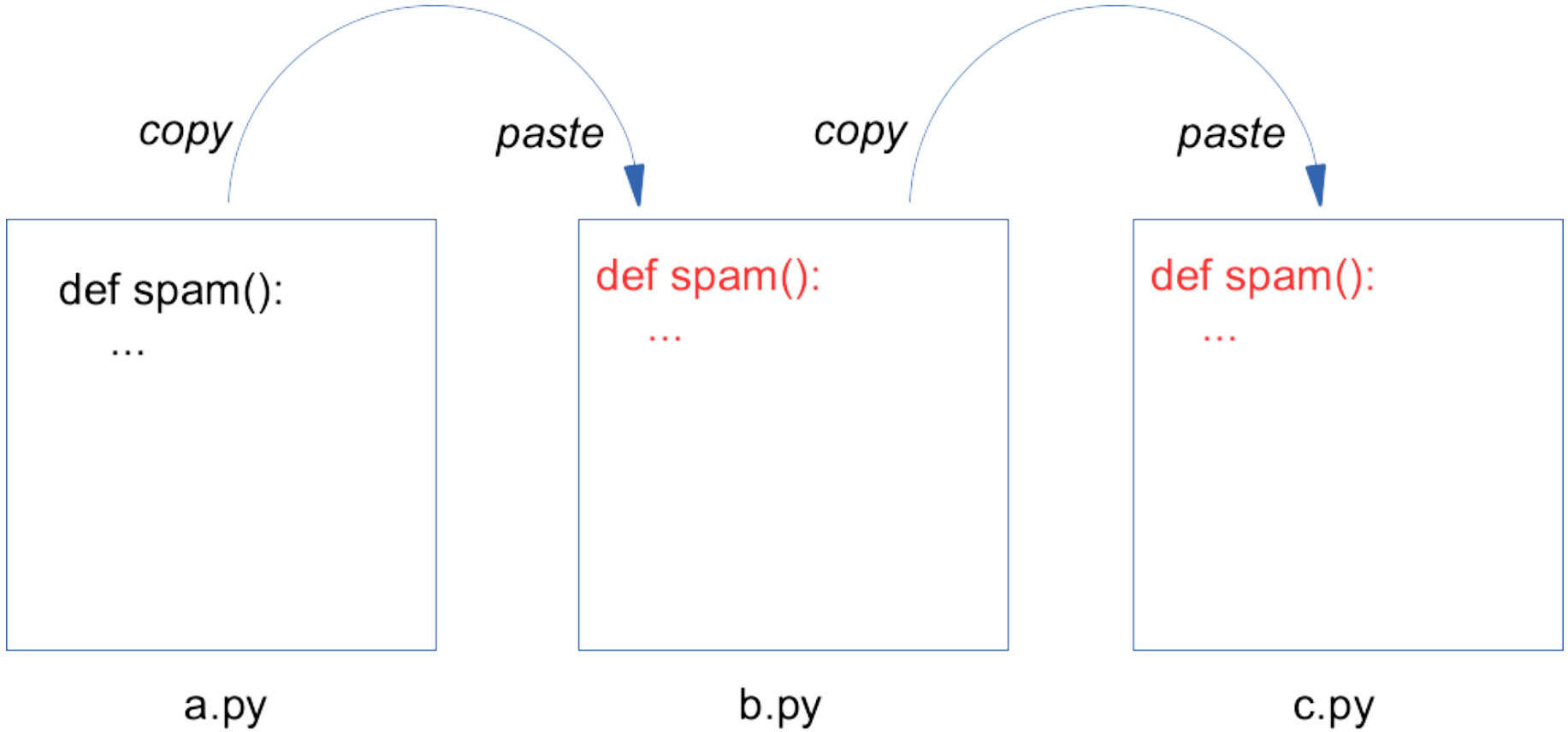


Copy/pasting functions

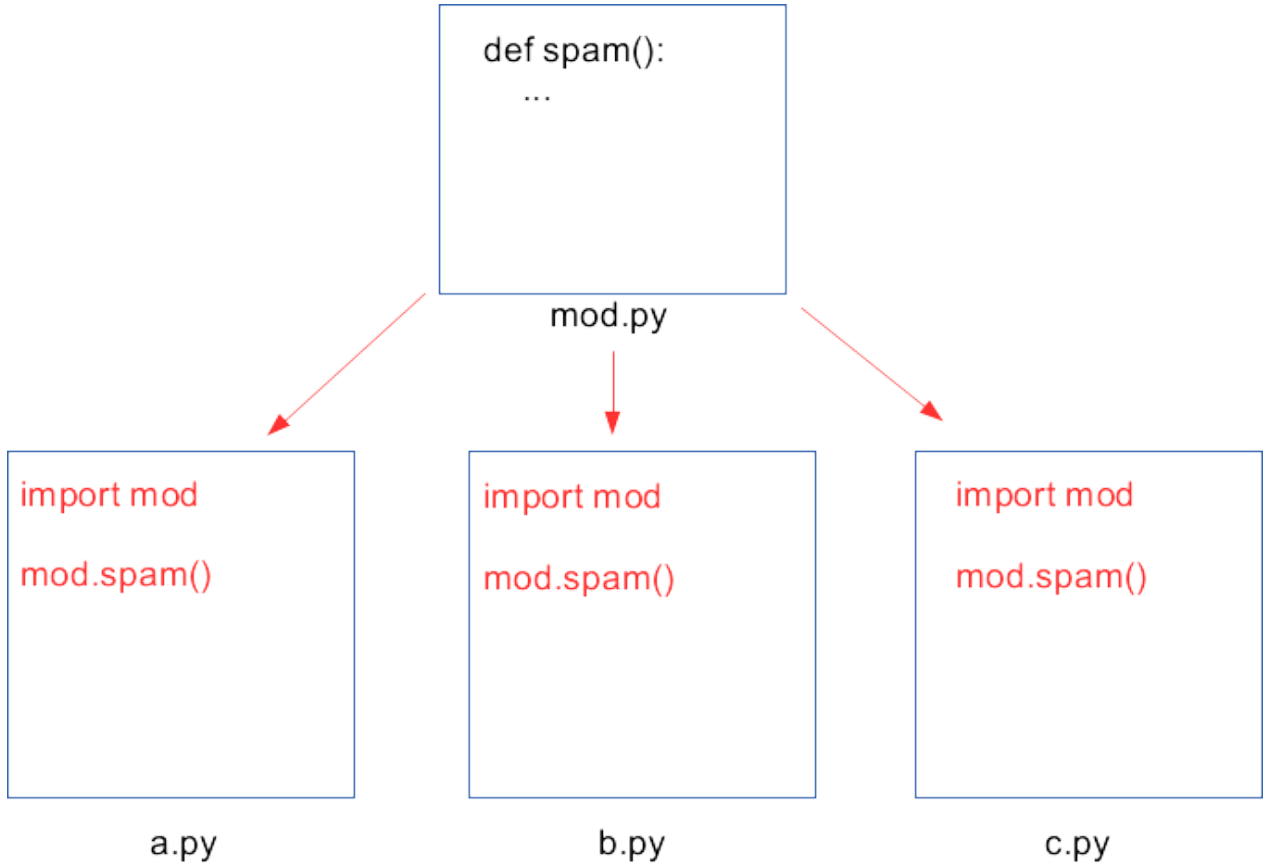


Copy/pasting functions

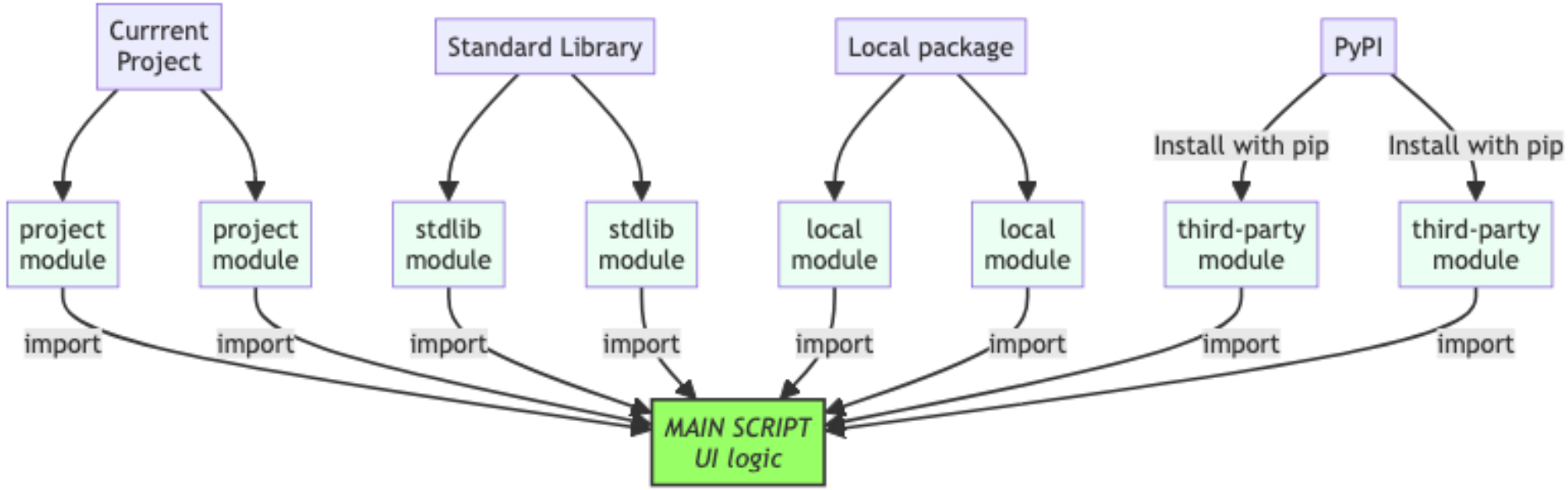
DON'T DO THIS!!



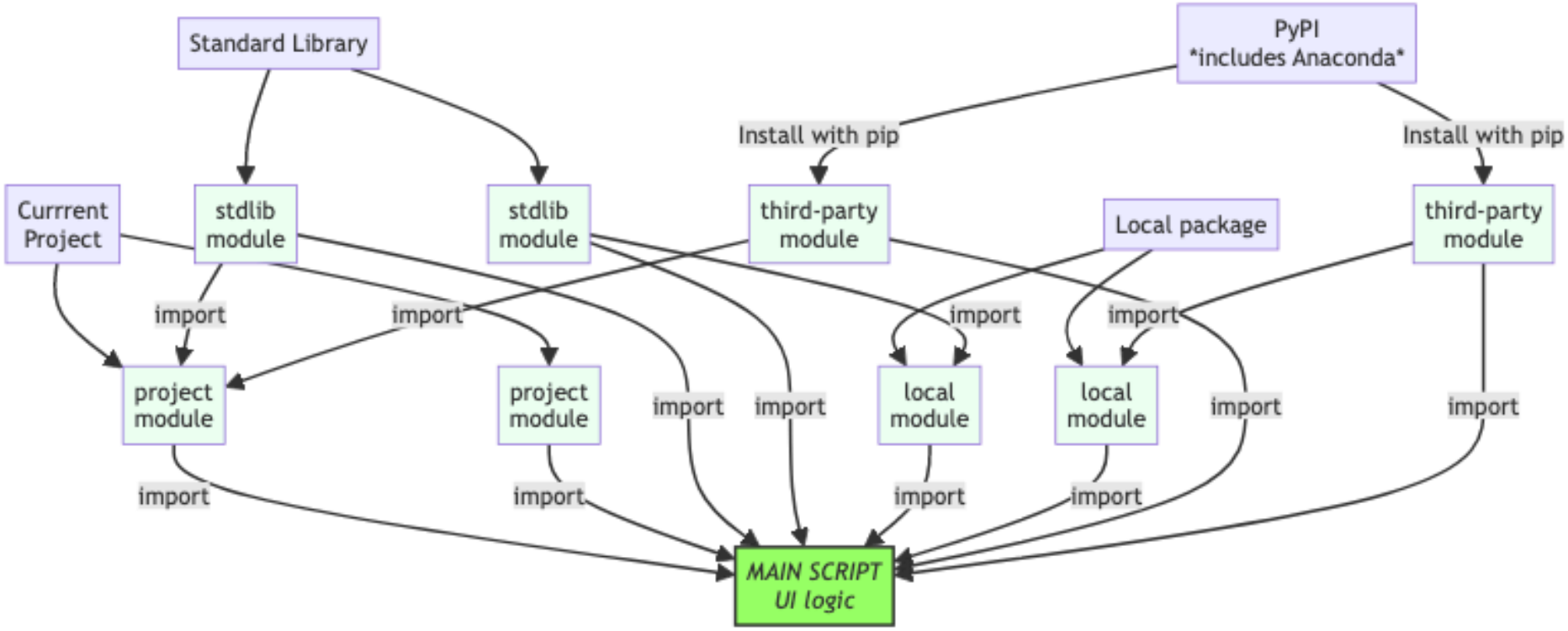
Using a module



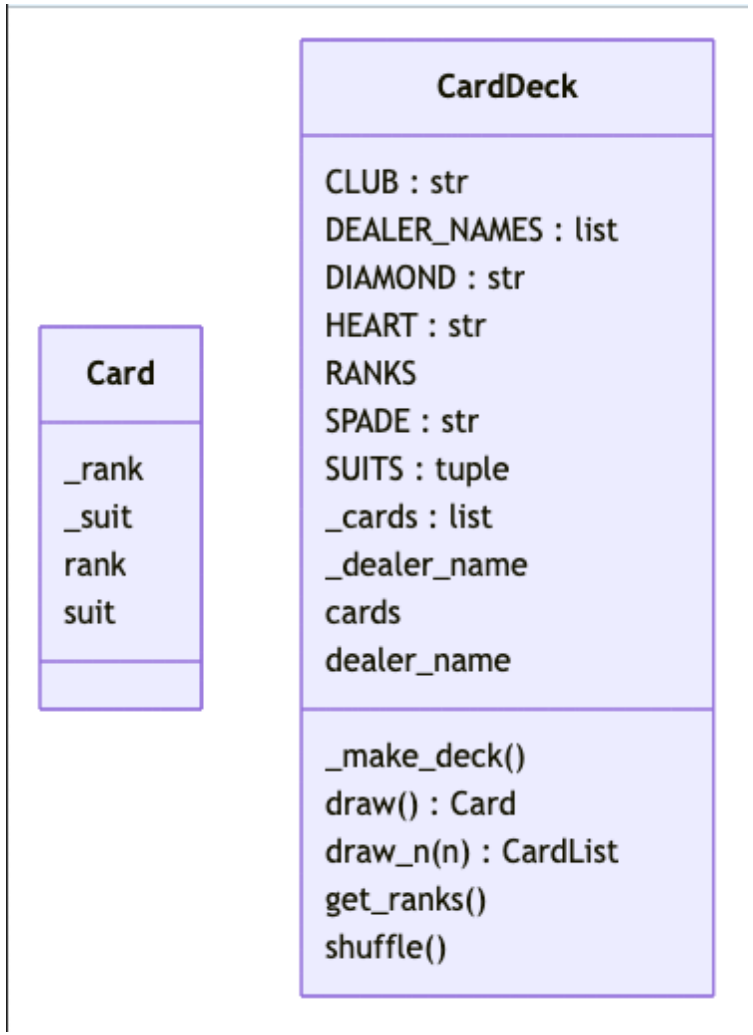
Project Imports



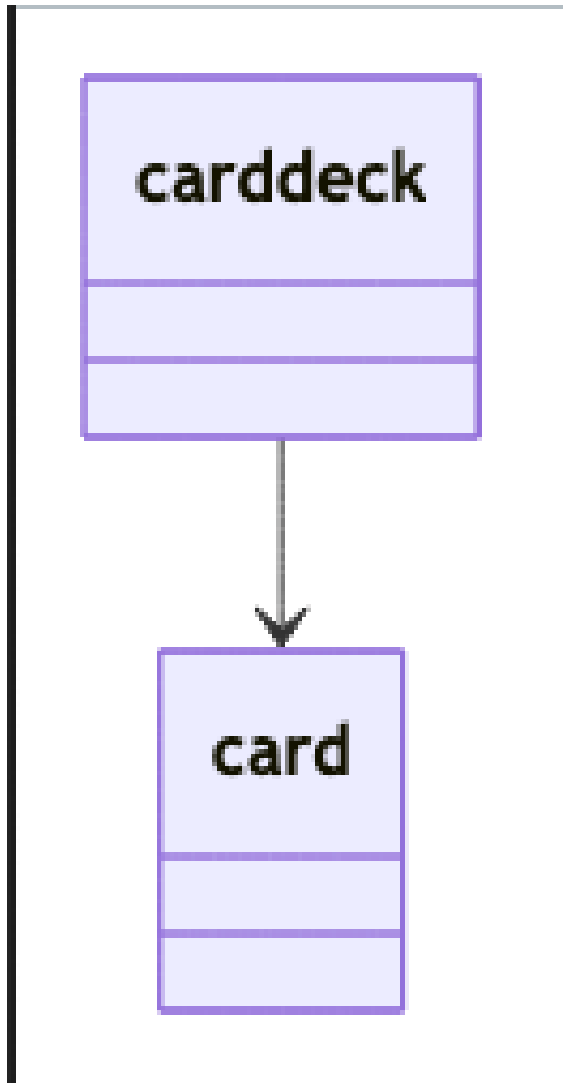
Project Imports (real-life)



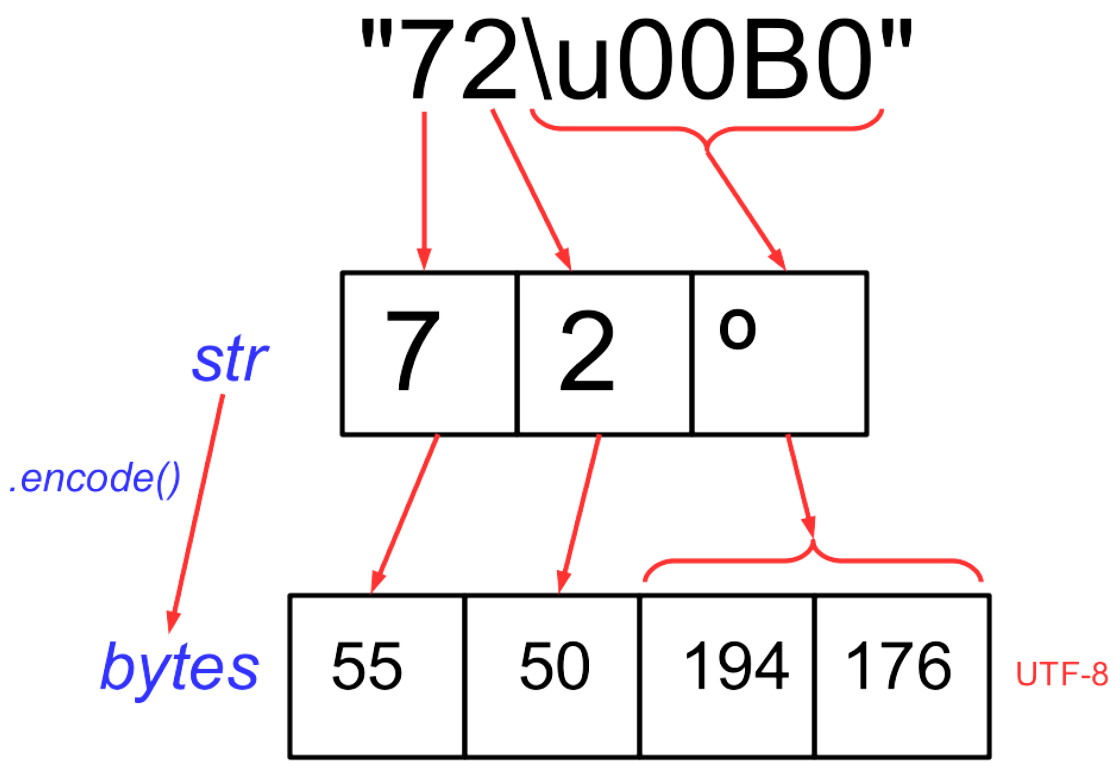
pyreverse (classes)



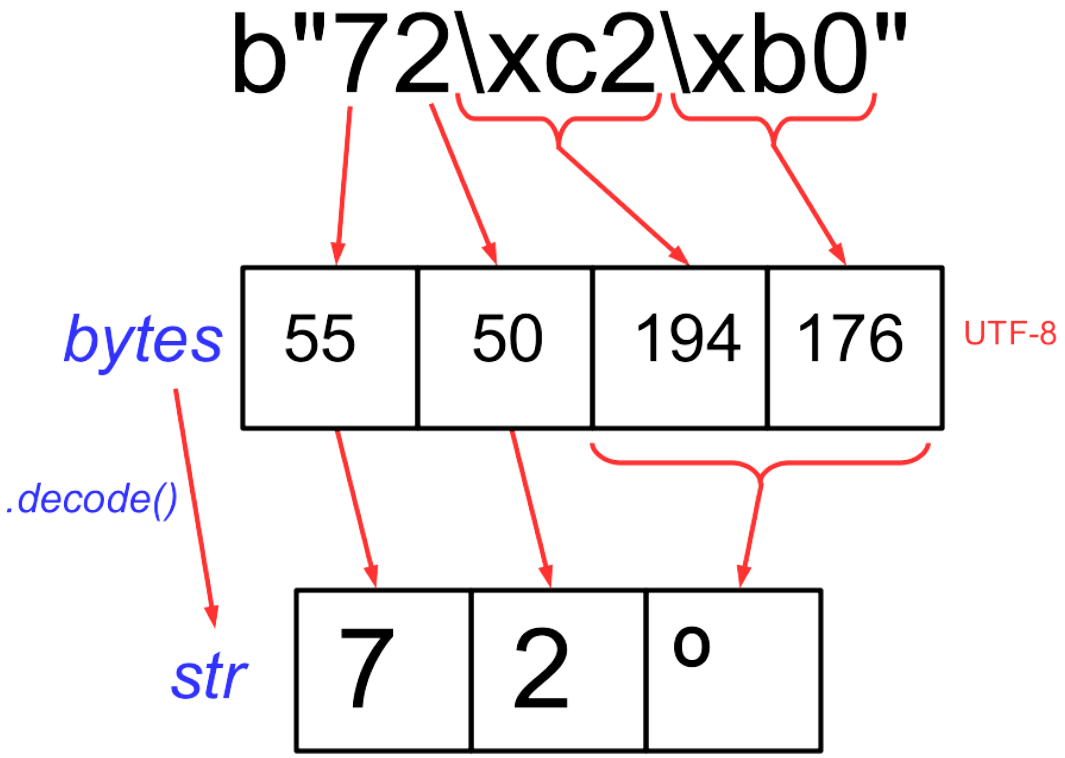
pyreverse (packages)



str to bytes



bytes to str



ElementTree

XML

```
<presidents>
    <president term="1">
        <first>George</first>
        <last>Washington</last>
    </president>
    <president term="2">
        <first>John</first>
        <last>Adams</last>
    </president>
</presidents>
```

ElementTree

```
Element
       tag="presidents"
   Element {"term": "1" }
      tag="president"
        Element
            tag="first"
            text="George"
        Element
            tag="last"
            text="Washington"
   Element {"term": "2" }
      tag="president"
        Element
            tag="first"
            text="John"
        Element
            tag="last"
            text="Adams"
```

Accessing Excel

- pandas.read_excel()
- openpyxl
- win32com (requires Excel to be running)
- CSV/TSV
- xlrd, xlwt, xlutil

Jupyter vs. IDE

Jupyter Notebook	IDE (VSCode, Spyder, Pycharm, …)
- Research - Exploratory - Experimental - Self-contained - Easy visualization - One file - Sharable ***Analyze data***	- Production-oriented - Structured - Modular - Share modules/packages - Development aids - Visualization is external - Manages many files (project) - Distributable ***Create scripts***

Pandas Location Accessors

Consistent access to rows, columns, and values

Select by *NAME* (column or index as string or number)

`.loc[ ]`

`.at[ ]`

Select by *POSITION* (column or row as integer)

`.iloc[ ]`

`.iat[ ]`

.loc[ ]

- Index/Column selector
  - single name `"spam"`
  - iterable of names `["spam", "ham", "eggs"]`
  - range of names `["spam":"toast"]`
  - boolean test/query `df_cust['state'] == 'VA'` *(rows only)*

```
df.loc[index-spec]                row(s) + all columns
df.loc[index-spec, column-spec]   rows(s) + column(s)
df.loc[:, column-spec]            all rows + column(s)
df.loc[:, df['col'] > 5]          all rows + column(s) with values > 5
```

.iloc[ ]

- position specification
  - single position 5
  - iterable of positions `[5, 6, 7, 8]`
  - range of positions `[5:8]`

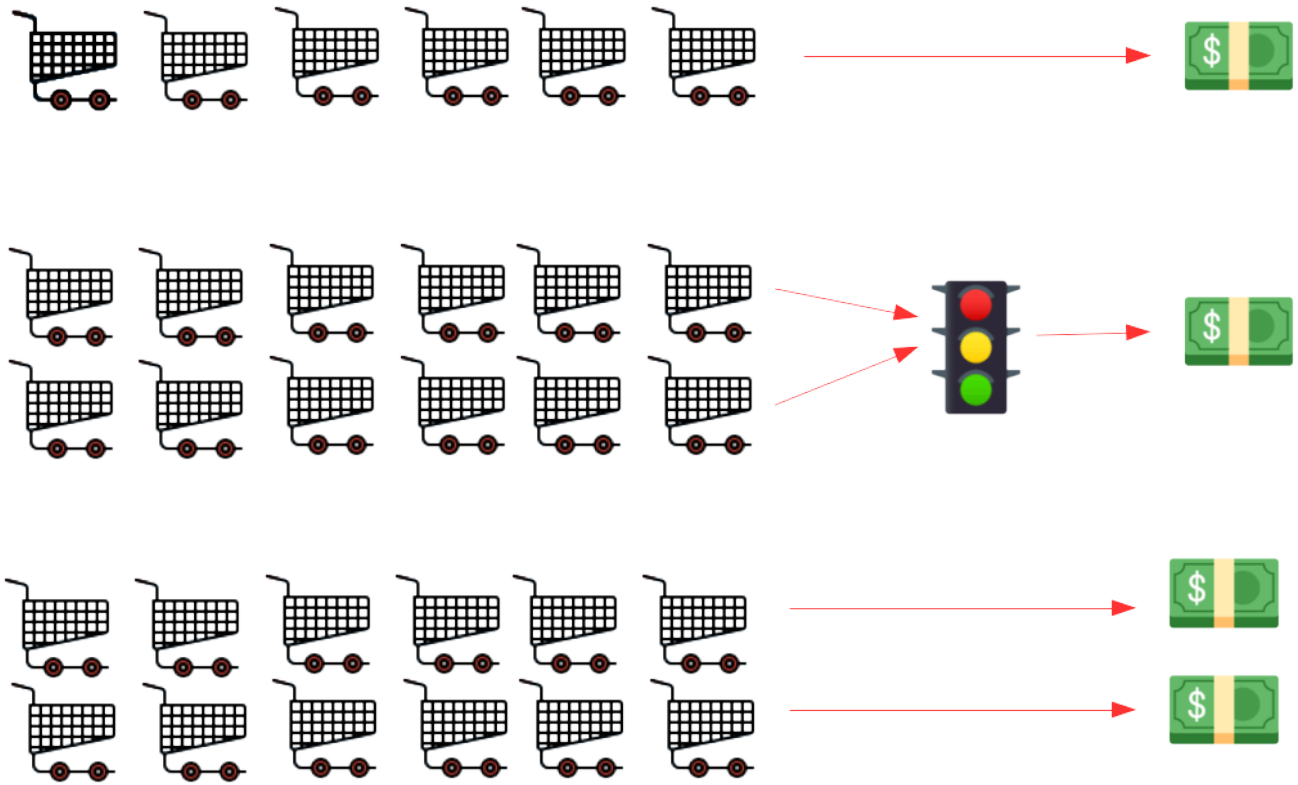
```
df.iloc[row-spec]                 row(s) + all columns
df.iloc[row-spec, column-spec]    rows(s) + column(s)
df.iloc[:, column-spec]           all rows + column(s)
```

.at[ ] & .iat[ ]

```
df.at[row-name, column-name]          value at row and column names

df.iat[row-position, column-position] value at row and column positions
```

Why sns?

Concurrency



HDF5 Overview